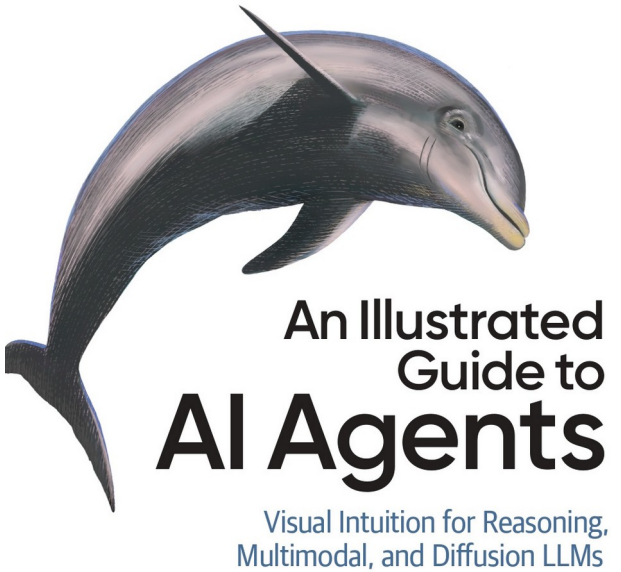
M5

Planning & ReAct

Module 5 — Making the agent autonomous



307307 — BI Methods for Generative AI

Agentic AI · University of Petra

Lecturer: _____ Date: _____

Learning Objectives

1. Explain task decomposition — breaking a goal into smaller steps.
2. Describe the ReAct loop: THOUGHT, ACTION, OBSERVATION, repeated until done.
3. Explain what makes an agent autonomous (a loop + a stopping rule).
4. Describe reflection (self-critique) and why it helps.
5. Explain agentic RAG: retrieval as a tool inside the ReAct loop, and how it differs from the fixed RAG pipeline of Module 3.
6. Recognise what lies beyond a single agent: evaluation, multi-agent, multimodal, coding.

Where We Are — the Final Piece

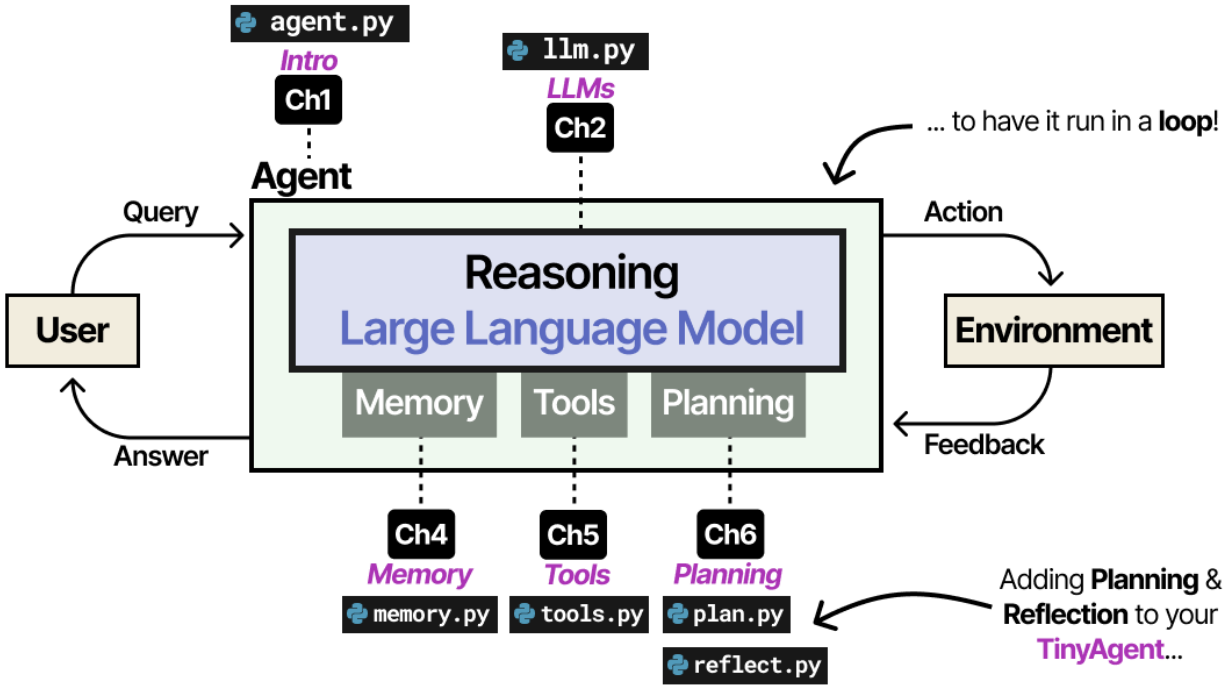


Figure from An Illustrated Guide to AI Agents (O'Reilly, 2026)

Think · Remember · Act

M2 - M4

Plan & loop (today)

M5 — autonomy

= a complete agent

the foundation is done

Planning — Decompose the Task

Real tasks take many steps.

Task decomposition means breaking a big goal into smaller, ordered sub-tasks, so the agent knows what to do next — and can revise the plan as it learns more.

A reasoning LLM does this naturally (“think step by step”).

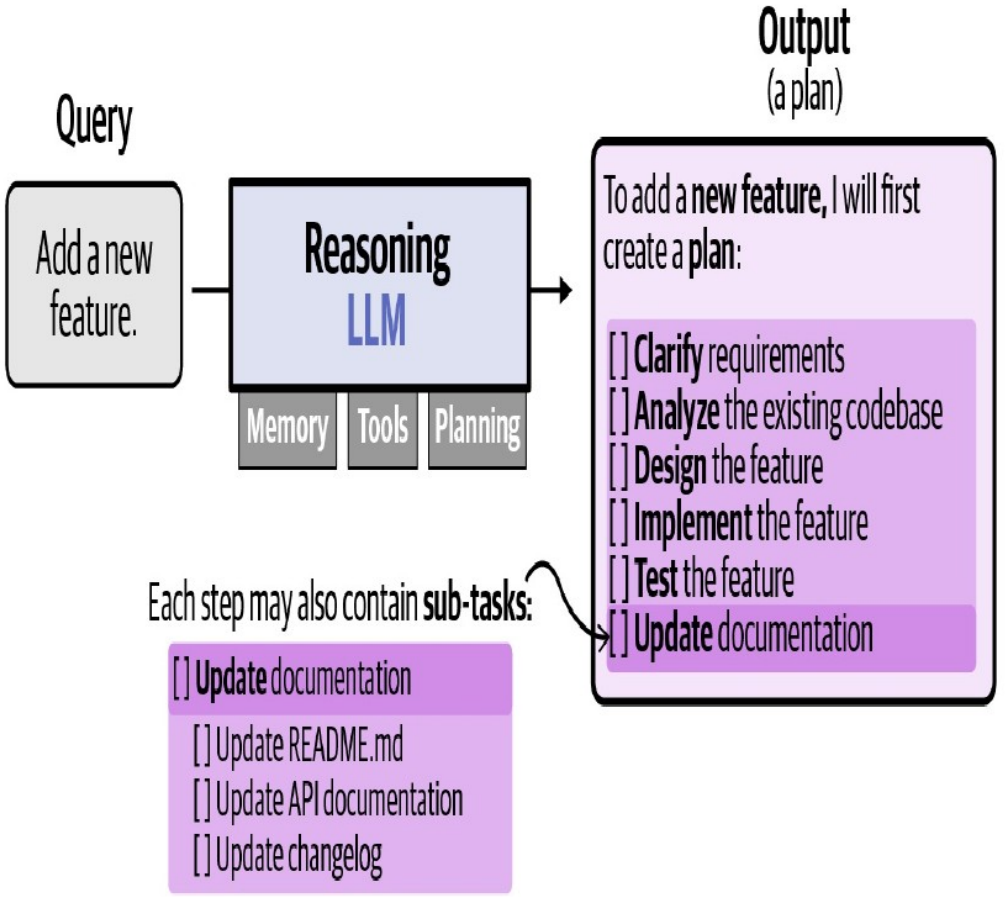


Figure 6-2, An Illustrated Guide to AI Agents (O'Reilly, 2026)

ReAct — Reason + Act in a Loop

The agent repeats a simple cycle until the task is done:

THOUGHT — reason about the next step

ACTION — call a tool (from Module 4)

OBSERVATION — we run it and feed the result back

...repeat, using each observation to decide the next thought, until a final answer.

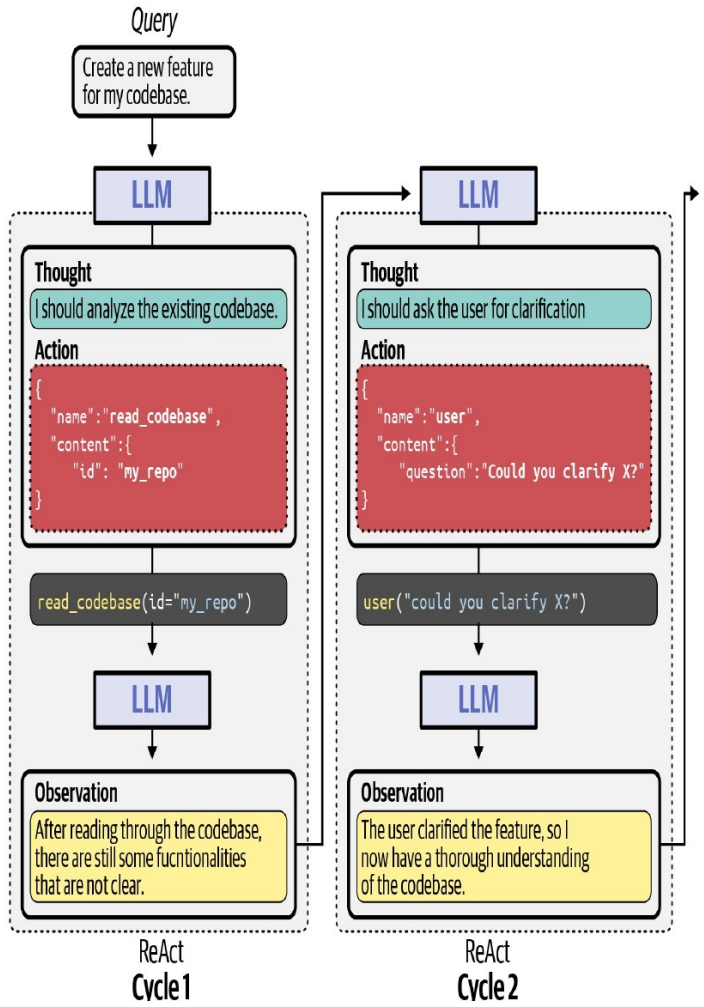


Figure 6-13 (O'Reilly, 2026)

What Makes It Autonomous

The “magic” is surprisingly simple — a loop. The agent keeps running THOUGHT -> ACTION -> OBSERVATION until it decides to stop.

A stopping rule

The agent signals it is done by calling a special final_answer.

max_steps

A cap on how many loops it may run — so a confused agent can never loop forever.

The complete single agent

Reasoning LLM (brain) + **Memory** (remembers its steps) + **Tools** (acts) + **Planning / ReAct** (sequences actions). Everything since Module 1 led here.

Reflection — Learning From Mistakes

Agents can get stuck or go wrong. Reflection has the agent critique its own work and course-correct.

Self-Refine

The LLM drafts, gives itself feedback, and refines — like a writer editing their own draft.

Reflexion

A richer version with separate roles: one acts, one evaluates, one writes feedback for the next try.

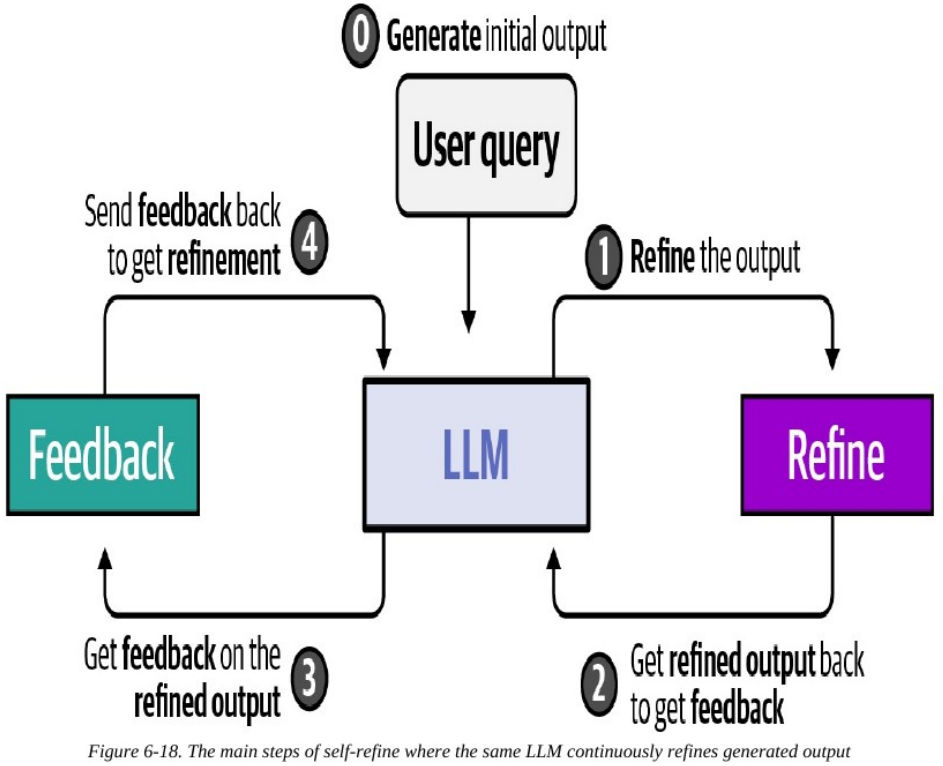


Figure 6-18. The main steps of self-refine where the same LLM continuously refines generated output

Figure 6-18 (self-refine), An Illustrated Guide to AI Agents (O'Reilly, 2026)

Agentic RAG — Retrieval as a Tool

In Module 3, RAG was a fixed pipeline: retrieve once, then generate. Agentic RAG puts retrieval inside the ReAct loop as one more tool.

RAG (Module 3)

The developer's code retrieves once, with the user's question as the search query. The model only sees what that one search returns.

Agentic RAG

The agent writes its own queries, decides whether to search again, and decides when it has enough to give the final answer.

Example: "When can I find the store manager?"

The question contains no name. Search 1 can find who the manager is; search 2 uses that name to find the working hours. A one-shot pipeline cannot know to run the second search.

Agentic RAG — What It Costs

Agentic RAG is not always better than RAG. It trades simplicity for flexibility.

What you gain

Searches that depend on earlier results, a choice between several knowledge sources, and a check that it has enough information before it answers.

What it costs

More LLM calls, so it is slower and more expensive, and less predictable: the agent can search badly or stop early. Read the log.

Start with a single agent

Each knowledge source can be its own tool, and the agent chooses between them. Specialised retrieval agents coordinated by a supervisor (multi-agent RAG) are more modular, but cost more and are harder to debug. Start simple.

What You Will Build Today

1 The ReAct loop

a for-loop of THOUGHT -> ACTION -> OBSERVATION.

2 Stopping & cap

final_answer to stop; max_steps so it can't loop forever.

3 Multi-step task

(4.6 + 6.685) x 4, then subtract 3.14 = 42.

4 Watch it reason

see the agent plan and solve, step by step.

5 Agentic RAG

Give the agent a search_docs tool, then compare its steps with one-shot RAG on a question that needs two lookups.

The climax of the build: no single tool could solve it — the loop does. That loop is the agent.

Beyond a Single Agent

A short survey of where the field goes next — the single-agent skills you built transfer directly.

The Road Ahead — at a Glance

Evaluation

Judge the outcome (did it get done?) and the step log (were the steps sound?), plus reliability and safety.

Multi-agent

Several specialised agents collaborate, often coordinated by a supervisor agent.

Multimodal

Agents that can also see and hear — images, audio, video — not just read text.

Coding agents

Agents that run, test, and fix real code in a dedicated environment.

Surveyed briefly — not separate build modules.

Key Takeaways

1. Planning decomposes a goal into ordered sub-tasks.
2. ReAct loops THOUGHT -> ACTION -> OBSERVATION until a final answer.
3. A loop + a stopping rule (max_steps) is what makes an agent autonomous.
4. Reflection lets an agent critique and improve its own work.
5. Agentic RAG = retrieval as a tool in the ReAct loop: the agent decides what to search for, and whether to search again.
6. Reasoning + Memory + Tools + Planning = a complete single agent.

Discussion

Q1 Name the three parts of a ReAct cycle and what each does.

Q2 What two things make an agent autonomous — and what does max_steps protect against?

Q3 Name the four components that together make a complete single agent.

Q4 How does agentic RAG differ from RAG? Give a question that needs more than one search.

That completes the agentic half — congratulations on building a full agent from scratch!